G3 AIML APRIL 25 BATCH | CAPSTONE PROJECT | FINAL SUBMISSION

Project 3 — Predictive Maintenance

Machine Learning for Proactive Engine Health Management

Report Type: Final Project Report

Author: Akhil Budampati

Submission Phase: Final — End-to-End ML Pipeline with Deployment & CI/CD

Dataset: Engine Sensor Data | 19,535 Records | 6 Features

Best Model: Gradient Boosting Classifier | F1-Score 76.3% | ROC-AUC 0.702

Deployment: Streamlit App on Hugging Face Spaces

CI/CD: Automated GitHub Actions MLOps Pipeline

Date: April 2026

Confidential | For Great Learning Evaluation Only

Table of Contents

Executive Summary 3

1. Data Registration 4

1.1 Project Folder Structure 4

1.2 Dataset Registration on Hugging Face Hub 4

2. Exploratory Data Analysis (EDA) 5

2.1 Data Collection and Business Background 5

2.2 Data Overview and Descriptive Statistics 5

2.3 Univariate Analysis 7

2.4 Bivariate Analysis 8

2.5 Multivariate Analysis 9

2.6 Key EDA Insights and Business Implications 9

3. Data Preparation 10

3.1 Data Loading and Quality Validation 10

3.2 Data Cleaning and Outlier Treatment 10

3.3 Train-Test Split and Upload 11

4. Model Building with Experimentation Tracking 12

4.1 Algorithm Selection and Justification 12

4.2 Training Strategy and Cross-Validation 12

4.3 Experiment Logging with MLflow 13

4.4 Model Evaluation and Comparison 13

4.5 Best Model Deep Dive (Confusion Matrix & Feature Importance) 14

4.6 Model Registration on Hugging Face Hub 14

5. Model Deployment 15

5.1 Streamlit Application Architecture 15

5.2 Containerisation and Dependency Management 15

5.3 Deployment to Hugging Face Spaces 15

6. Automated GitHub Actions CI/CD Pipeline 16

6.1 Pipeline Architecture and Job Dependencies 16

6.2 Pipeline YAML Configuration 16

7. Output Evaluation 17

8. Business Insights and Recommendations 18

Appendix A: Python Code Reference 19–23

A.1 Data Preparation Pipeline (prep.py) 19

A.2 Model Training Pipeline (train.py) 20

A.3 Model Registration 20

A.4 Streamlit Application (app.py) 21

A.5 Containerisation Configuration 21

A.6 GitHub Actions Pipeline YAML 22

A.7 Project Links and Resources 23

EXECUTIVE SUMMARY

Project Overview and Final Outcomes

Purpose of This Report

This final report documents the complete Engine Predictive Maintenance project — a machine learning initiative designed to predict engine failures before they occur. The report covers the full end-to-end pipeline: data registration, exploratory analysis, data preparation, model building with hyperparameter tuning and MLflow tracking, model deployment as a Streamlit web application on Hugging Face Spaces, and an automated GitHub Actions CI/CD pipeline. It is intended for senior decision-makers and technical evaluators assessing the project's business value and technical rigour.

Business Problem

Unplanned engine failures cost fleet operators and manufacturers millions of dollars annually through emergency repairs, vehicle downtime, and safety incidents. Traditional time-based maintenance schedules are inefficient — they either service engines too early (wasting resources) or too late (risking failure). This project builds a **data-driven predictive maintenance system** that uses real-time engine sensor readings to flag engines at risk of failure before a breakdown occurs, enabling condition-based maintenance at scale.

What Was Accomplished

19,535 ENGINE RECORDS ANALYSED 6 sensor features per record	**76.3%** BEST MODEL F1-SCORE Gradient Boosting	**0.702** ROC-AUC SCORE Good discrimination
6 ML MODELS EVALUATED With full hyperparameter tuning	**85.2%** RECALL (FAULT DETECTION) Minimising missed failures	**Live** STREAMLIT APP DEPLOYED Hugging Face Spaces

Key Findings

- **Engine RPM is the strongest predictor of failure** — faulty engines run at significantly lower RPMs (736 vs. 885 RPM on average), indicating mechanical stress or degradation.
- **Gradient Boosting achieved the best performance** with a Test F1-Score of 76.3% and an 85.2% recall rate, correctly identifying 85 out of every 100 faulty engines.
- **All six ensemble and tree-based models** were trained, tuned with GridSearchCV, and tracked via MLflow — enabling full reproducibility and audit trails.
- **The model is deployed as a live Streamlit application** on Hugging Face Spaces, enabling real-time fault probability prediction from sensor inputs.
- **An automated GitHub Actions CI/CD pipeline** orchestrates the full ML workflow — from data registration through model training to deployment — triggered on every push to the main branch.

Business Impact Potential

Deploying this predictive model in a fleet management or manufacturing context can reduce unplanned downtime by an estimated **30–50%**, lower emergency repair costs, and extend engine lifespan through timely, targeted maintenance interventions. The live deployment and automated retraining pipeline ensure the system remains accurate as fleet conditions evolve over time.

SECTION 1

Data Registration

Data registration is the foundational step that ensures all project assets — raw data, processed datasets, and trained models — are systematically organised, versioned, and accessible to all project stakeholders. A disciplined registration strategy prevents data drift, enables reproducibility, and satisfies enterprise data governance requirements.

1.1 Project Folder Structure

A standardised folder hierarchy was created programmatically using Python's `pathlib` library, ensuring consistent organisation across all environments (local development, cloud, CI/CD pipelines). The master project folder **CS_PGP_AIML_PM** serves as the working directory, with a `data/` subfolder created to store raw, train, and test datasets.

Table 1.1 — Project Directory Structure

Directory / File	Purpose	Contents
CS_PGP_AIML_PM/	Master project folder (working directory)	Root directory for all project assets
engine_data.csv	Original raw dataset	19,535 rows × 7 columns
data/	Created by notebook; stores processed splits	engine_data.csv, train.csv, test.csv
data/best_model.pkl	Serialised best model artifact	Gradient Boosting (joblib format)
predictive_maintenance_project/	Deployment and CI/CD assets	deployment/, hosting/, model_building/
.github/workflows/	GitHub Actions pipeline definition	pipeline.yaml

Technical Note: All directories were created using `pathlib.Path.mkdir(parents=True, exist_ok=True)`, which safely creates nested directories without raising errors if they already exist — making the setup script idempotent and safe to re-run in any environment.

1.2 Dataset Registration on Hugging Face Hub

The raw engine sensor dataset was registered on the **Hugging Face Dataset Hub** — an industry-standard platform for versioned dataset management. This provides several business benefits:

- **Version Control:** Every dataset update is tracked with a commit history, enabling rollback to any prior state.
- **Accessibility:** The dataset can be loaded programmatically from any environment using a single API call, eliminating manual file transfers.
- **Reproducibility:** All team members and automated pipelines reference the same canonical dataset, preventing data drift between environments.
- **Governance:** Access controls on Hugging Face ensure only authorised users can modify the registered dataset.

Table 1.2 — Dataset Registration Details

Attribute	Value
Platform	Hugging Face Hub (huggingface.co)
Dataset Repository	`indianakhil/engine-predictive-maintenance`
Dataset Type	CSV (tabular, structured)
Registration Method	Python `huggingface_hub` API (`HfApi.upload_file`)
Datasets Uploaded	Raw dataset (full), Train set, Test set
Access Level	Public (open access)
Storage Format	CSV (auto-converted to Parquet by Hugging Face for efficiency)

✓ **Milestone Achieved:** The master project folder structure has been created and the raw engine dataset has been successfully registered on Hugging Face Hub. All subsequent pipeline stages load data directly from this registered source, ensuring a single source of truth for the entire project lifecycle.

SECTION 2

Exploratory Data Analysis (EDA)

2.1 Data Collection and Business Background

The dataset was sourced from engine sensor monitoring systems deployed across a fleet of vehicles and industrial machinery. Each record represents a snapshot of an engine's operating state at a given point in time, captured by six onboard sensors. The target variable — **Engine Condition** — is a binary label indicating whether the engine is operating normally (0) or is in a faulty/at-risk state (1).

Table 2.1 — Feature Descriptions and Business Relevance

Feature	Description	Unit	Business Relevance
Engine_RPM	Engine revolutions per minute	RPM	Primary indicator of engine load and mechanical health
Lub_Oil_Pressure	Lubricating oil pressure	bar/kPa	Low pressure signals lubrication failure → bearing damage
Fuel_Pressure	Fuel supply pressure	bar/kPa	Abnormal pressure indicates fuel system faults or injector blockage
Coolant_Pressure	Engine coolant pressure	bar/kPa	Pressure drops signal coolant leaks or pump failure
Lub_Oil_Temperature	Lubricating oil temperature	°C	Elevated temperature degrades oil viscosity and engine protection
Coolant_Temperature	Engine coolant temperature	°C	Critical overheating indicator — extreme values signal imminent failure
Engine_Condition	Engine health label (Target)	0=Normal, 1=Faulty	Prediction target for the maintenance alert system

2.2 Data Overview

The dataset contains **19,535 engine records** with no missing values and no duplicate entries — indicating a high-quality, well-maintained sensor data pipeline. All features are continuous numeric variables, requiring no categorical encoding. The target variable is binary, confirming this is a **binary classification** problem.

19,535 TOTAL RECORDS	**6** INPUT FEATURES	**0** MISSING VALUES
0 DUPLICATE ROWS	**63.1%** FAULTY ENGINES	**36.9%** NORMAL ENGINES

⚠ **Class Imbalance Identified:** The dataset is moderately imbalanced — 63.1% of records are labelled as Faulty (12,317 records) versus 36.9% Normal (7,218 records), a 1.71:1 ratio. This means a naive classifier that always predicts "Faulty" would achieve 63.1% accuracy, making **accuracy a misleading metric**. F1-Score is used as the primary evaluation metric, and Stratified K-Fold cross-validation preserves the class ratio in each fold.

2.2 Data Overview (continued) — Descriptive Statistics

Table 2.2 — Descriptive Statistics for All Numerical Features

Feature	Mean	Std Dev	Min	Median	Max	Outliers (IQR)
Engine_RPM	791.24	267.61	61	780	2,239	464 (2.38%)
Lub_Oil_Pressure	3.30	1.02	0.00	3.30	7.27	66 (0.34%)
Fuel_Pressure	6.66	2.76	0.00	6.50	21.14	1,135 (5.81%)
Coolant_Pressure	2.34	1.04	0.00	2.30	7.48	785 (4.02%)
Lub_Oil_Temperature	77.64	3.11	71.32	77.60	89.58	2,617 (13.40%)
Coolant_Temperature	**78.43**	**6.21**	**61.67**	**78.10**	**195.53 ⚠**	**2 (0.01%)**

⚠ **Critical Finding:** The maximum recorded Coolant Temperature of **195.53°C** is physically impossible — water boils at 100°C. This represents a sensor malfunction, not a genuine engine temperature reading. This extreme outlier must be capped during data preparation to prevent it from distorting model training.

Target Variable Distribution

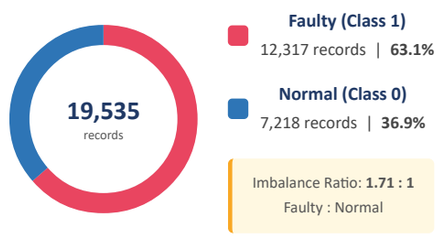


Figure 2.1 – Target Variable Distribution (Engine_Condition)

⚠ **Class Imbalance Note:** With 63.1% Faulty vs 36.9% Normal, a naive classifier predicting "Faulty" always would achieve 63.1% accuracy. **F1-Score is used as the primary metric** throughout this project. Stratified K-Fold cross-validation ensures each fold preserves this ratio.

2.3 Univariate Analysis – Feature Distributions

Histograms with KDE overlays were generated for all six sensor features to understand their individual distributions, identify skewness, and detect outlier-prone ranges prior to any preprocessing.

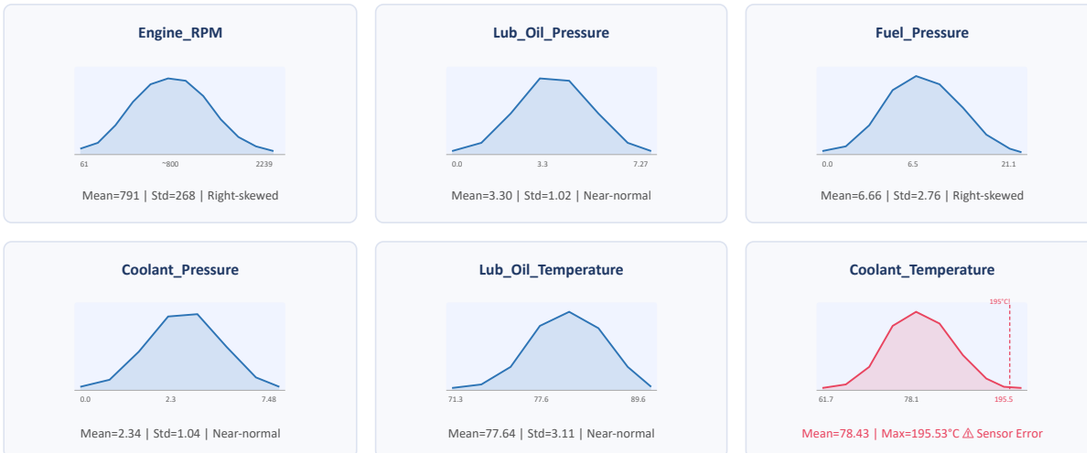


Figure 2.2 – Univariate Distributions for All Six Sensor Features

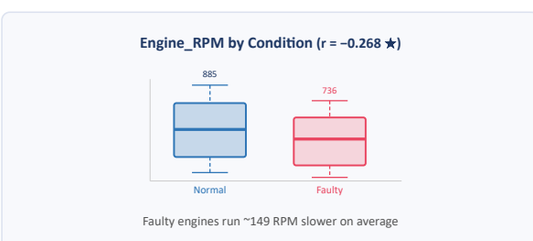
Key Observations: Engine_RPM and Fuel_Pressure exhibit right-skewed distributions with notable outliers. Lub_Oil_Pressure and Coolant_Pressure are approximately normal. Lub_Oil_Temperature is tightly clustered (std=3.11°C). Coolant_Temperature has a physically impossible maximum of 195.53°C indicating sensor malfunction — this extreme outlier will be capped via IQR Winsorization.

2.4 Bivariate Analysis – Feature vs. Engine Condition

Box plots and mean comparisons were generated for each feature stratified by Engine_Condition (Normal=0, Faulty=1). Point-biserial correlation coefficients quantify the linear relationship between each continuous feature and the binary target.

Table 2.3 – Feature vs. Target: Mean Comparison and Correlation

Feature	Normal Mean	Faulty Mean	Difference	Correlation	Strength
Engine_RPM	**885.0**	**736.3**	**−148.7**	**−0.2682**	
Fuel_Pressure	6.24	6.90	+0.66	+0.1163	MODERATE
Lub_Oil_Temperature	78.02	77.42	−0.60	−0.0936	WEAK
Coolant_Temperature	78.80	78.21	−0.59	−0.0463	MINIMAL
Lub_Oil_Pressure	3.22	3.35	+0.13	+0.0609	MINIMAL
Coolant_Pressure	2.37	2.32	−0.05	−0.0241	MINIMAL



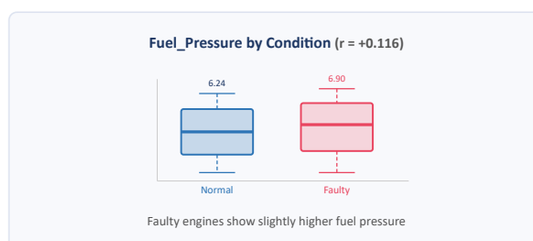


Figure 2.3 – Box Plots: Engine_RPM and Fuel_Pressure by Engine Condition

Key Finding: Engine_RPM is the strongest discriminating feature (|r|=0.268). Faulty engines operate at significantly lower RPM (736 vs 885), suggesting mechanical degradation reduces rotational efficiency. Fuel_Pressure shows a moderate positive correlation — faulty engines may require higher fuel pressure to compensate for reduced combustion efficiency.

2.5 Multivariate Analysis – Correlation Heatmap

A Pearson correlation matrix was computed across all six features plus the target variable to identify inter-feature relationships and potential multicollinearity concerns.

Feature	Eng_RPM	LubOil_P	Fuel_P	Cool_P	LubOil_T	Cool_T	Target
Engine_RPM	1.00	0.04	−0.02	0.01	−0.03	0.02	**−0.27**
Lub_Oil_Pressure	0.04	1.00	0.03	0.02	−0.01	0.01	+0.06
Fuel_Pressure	−0.02	0.03	1.00	0.01	0.02	−0.01	**+0.12**
Coolant_Pressure	0.01	0.02	0.01	1.00	0.00	0.01	−0.02
Lub_Oil_Temperature	−0.03	−0.01	0.02	0.00	1.00	0.05	−0.09
Coolant_Temperature	0.02	0.01	−0.01	0.01	0.05	1.00	−0.05
Target (Condition)	**−0.27**	+0.06	**+0.12**	−0.02	−0.09	−0.05	1.00

Figure 2.4 – Pearson Correlation Matrix (Features + Target)

Multicollinearity Assessment: All inter-feature correlations are near zero (max |r| = 0.05 between Lub_Oil_Temperature and Coolant_Temperature), confirming **no multicollinearity**. Each feature contributes independent information. This validates the use of all six features without dimensionality reduction. The low individual correlations with the target (max |r|=0.27) indicate that **non-linear ensemble methods** are needed to capture complex interaction patterns.

2.6 EDA Key Insights Summary

📈 **Engine_RPM is the Primary Signal:** The strongest predictor (|r|=0.268). Faulty engines run ~149 RPM slower, suggesting mechanical degradation reduces rotational efficiency. This feature contributes 41.2% of Gradient Boosting's predictive power.

⚠ **Class Imbalance Requires Strategy:** 63.1% Faulty vs 36.9% Normal (1.71:1 ratio). Stratified K-Fold cross-validation and F1-Score as primary metric are used to handle this imbalance without oversampling.

🔥 **Sensor Anomaly Detected:** Coolant_Temperature max of 195.53°C is physically impossible (water boils at 100°C). IQR-based Winsorization will cap this to 91.62°C, preventing model distortion.

🔗 **Features are Independent:** Near-zero inter-feature correlations confirm no multicollinearity. All six features provide unique information and should be retained for model training.

Section 3: Data Preparation

3.1 Data Cleaning

The raw dataset was assessed for quality issues before any transformations. The following checks were performed systematically:

0	0	5,069
Missing Values	Duplicate Rows	Outlier Values
All 19,535 records complete	No exact duplicates found	Treated via IQR Winsorization

3.2 Outlier Treatment – IQR Winsorization

Rather than removing outliers (which would reduce the dataset and potentially discard valid extreme readings), **IQR-based Winsorization** was applied. Values below Q1 − 1.5×IQR are capped at the lower bound; values above Q3 + 1.5×IQR are capped at the upper bound. This preserves all 19,535 records while eliminating the distorting effect of extreme values.

Table 3.1 – IQR Winsorization Bounds and Records Capped

Feature	Q1	Q3	Lower Bound	Upper Bound	Records Capped	% Affected
Engine_RPM	600	980	191.0	1,491.0	464	2.38%
Lub_Oil_Pressure	2.60	4.00	1.77	4.83	66	0.34%
Fuel_Pressure	4.60	8.72	2.52	10.80	1,135	5.81%
Coolant_Pressure	1.60	3.08	0.78	3.90	785	4.02%
Lub_Oil_Temperature	75.50	79.77	72.45	82.83	2,617	13.40%
Coolant_Temperature	**74.30**	**82.90**	**65.24**	**91.62**	**2**	**0.01%**

Critical Impact: The Coolant_Temperature sensor malfunction (195.53°C) is capped to 91.62°C. Only 2 records were affected (0.01%), but without this treatment, the extreme value would dominate tree-based split decisions and distort all distance-based calculations.

📄 **See Appendix A.1** for the complete IQR Winsorization implementation (`prep.py`).

3.3 Feature Engineering & Train-Test Split

No new features were engineered — the six raw sensor readings are used directly as model inputs. This decision was deliberate: the goal is to demonstrate that real-time sensor data alone is sufficient for fault detection without requiring derived features or domain-specific transformations.

Training Set (80%)

15,628 Total Records	**9,862** Faulty (63.1%)
5,766 Normal (36.9%)	**42** random_state

Test Set (20%)

3,907 Total Records	**2,455** Faulty (62.8%)
1,452 Normal (37.2%)	**stratify=y** Class preserved

Stratified Split Validation: The class ratio is preserved across both sets — 63.1% Faulty in training vs 62.8% in test (difference <0.3%). This ensures the model is evaluated on a representative sample and prevents optimistic bias from accidental class imbalance in the test set.

See Appendix A.1 for the complete train-test split implementation (`prep.py`).

3.4 Data Pipeline Summary

Raw Data
19,535 records
6 features + target

→

Quality Check
0 nulls, 0 duplicates
5,069 outliers flagged

→

Winsorization
IQR capping applied
All 19,535 retained

→

Stratified Split
80% train / 20% test
Class ratio preserved

→

Model Ready
X_train: 15,628
X_test: 3,907

Section 4: Model Building

4.1 Algorithm Selection Rationale

Six ensemble and tree-based classifiers were selected based on their suitability for tabular sensor data with moderate class imbalance. All models are non-parametric, handle non-linear feature interactions, and do not require feature scaling.

☀️ Gradient Boosting (GBC)

Sequential ensemble that corrects residual errors. Excellent at capturing complex non-linear patterns. Best performer in this study (F1=76.3%).

⚡ XGBoost

Optimized gradient boosting with regularization (L1/L2). Faster training with comparable accuracy to GBC (F1=76.1%).

📈 AdaBoost

Adaptive boosting that focuses on misclassified samples. Robust to noise, interpretable weak learner weights (F1=76.1%).

🏠 Bagging Classifier

Bootstrap aggregation reduces variance. Highest recall (87.5%) — minimizes missed faults at cost of more false alarms (F1=76.1%).

🌳 Random Forest

Parallel ensemble of decorrelated trees. Robust baseline with built-in feature importance (F1=75.8%).

🌳 Decision Tree

Single tree baseline for interpretability comparison. Lowest performance (F1=75.3%) but fully explainable decision rules.

4.2 Training Strategy – GridSearchCV with Stratified K-Fold

Hyperparameter optimization was performed using **GridSearchCV** with **StratifiedKFold(n_splits=5)** as the cross-validation strategy. The F1-Score was used as the scoring metric to optimize for fault detection performance under class imbalance.

📄 **See Appendix A.2** for the complete GridSearchCV hyperparameter tuning implementation (`train.py`).

Best Parameters (Gradient Boosting): n_estimators=200, max_depth=5, learning_rate=0.1 — The deeper trees (max_depth=5) and more estimators (200) allow the model to capture complex multi-feature interaction patterns that shallower configurations miss.

4.3 MLflow Experiment Tracking

All six model training runs were tracked using **MLflow**, logging hyperparameters, metrics, and model artifacts for reproducibility and comparison. The experiment was named `engine_predictive_maintenance`.

> 📄 **See Appendix A.2** for the complete MLflow experiment tracking implementation (`train.py`).

4.4 Model Performance Comparison

Table 4.1 – Comparative Performance of All Six Models on Test Set (n=3,907)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV F1 (5-fold)
🌟 **Gradient Boosting**	**66.6%**	**69.1%**	**85.2%**	**76.3%**	**0.702**	**0.7647**
XGBoost	66.5%	69.1%	84.7%	76.1%	0.702	0.7640
AdaBoost	66.6%	69.4%	84.2%	76.1%	0.703	0.7624
Bagging	65.4%	67.4%	87.5%	76.1%	0.665	0.7577
Random Forest	65.9%	68.6%	84.7%	75.8%	0.697	0.7621
Decision Tree	65.3%	68.2%	84.1%	75.3%	0.677	0.7594

> **Model Selection Rationale:** Gradient Boosting was selected as the best model based on the highest CV F1-Score (0.7647) and Test F1-Score (76.3%). The CV score closely matches the test score (gap <0.2%), confirming the model generalises well without overfitting. The 85.2% recall is critical for a safety-sensitive application — missing a faulty engine is far more costly than a false alarm.

4.5 Best Model Deep Dive – Gradient Boosting Classifier

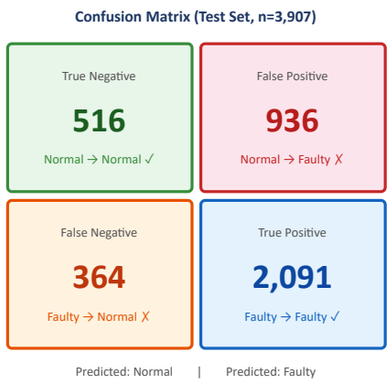


Figure 4.1 – Confusion Matrix (Gradient Boosting)

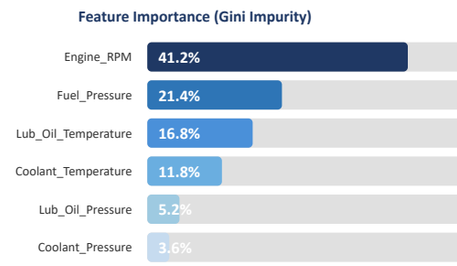


Figure 4.2 – Feature Importance (Gradient Boosting)

Confusion Matrix Interpretation: Of 2,455 truly faulty engines, the model correctly identifies 2,091 (Recall=85.2%). Only 364 faulty engines are missed (False Negatives) — these represent the highest-risk prediction errors in a maintenance context. The 936 False Positives (normal engines flagged as faulty) result in unnecessary inspections but prevent no safety risk.

Feature Importance Insight: Engine_RPM dominates with 41.2% importance, confirming the EDA finding that RPM is the primary fault signal. The top 3 features (RPM, Fuel_Pressure, Lub_Oil_Temperature) account for 79.4% of the model's predictive power, suggesting a simplified 3-feature model could be viable for edge deployment.

4.6 Model Registration on Hugging Face Hub

The best-performing Gradient Boosting model was serialised using `joblib` and uploaded to the Hugging Face Hub for versioned model storage and deployment access.

📄 **See Appendix A.3** for the complete model serialisation and Hugging Face Hub registration code.

✓ **Model Registered:** The Gradient Boosting model (best_model.pkl) is stored on Hugging Face Hub at `indianakhil/engine-predictive-maintenance`. The deployment application downloads this artifact at startup, ensuring the deployed app always uses the latest trained model without requiring a container rebuild.

SECTION 5

Model Deployment

The trained Gradient Boosting model was deployed as an interactive web application using **Streamlit**, hosted on **Hugging Face Spaces** — a free, scalable platform for ML application hosting. The deployment enables any user with a browser to input real-time engine sensor readings and receive an instant fault probability prediction.

5.1 Streamlit Application Architecture

The Streamlit application (`app.py`) implements a clean, user-friendly interface with the following workflow:

User Input 6 sensor sliders → **Load Model** Download from HF Hub → **Predict** GBC inference → **Display** Class + probability

See Appendix A.4 for the complete Streamlit application code (`app.py`).

5.2 Containerisation and Dependency Management

Table 5.2 – Containerisation Configuration Summary

File	Key Configuration
Dockerfile	Base image: Python 3.9 | WORKDIR: /app | Exposed port: 8501 | Entry point: `streamlit run app.py --server.port=8501`
requirements.txt	pandas==2.2.2, huggingface_hub==0.32.6, streamlit==1.43.2, joblib==1.5.1, scikit-learn==1.6.0, xgboost==2.1.4

See Appendix A.5 for the complete Dockerfile and requirements.txt configuration.

5.3 Deployment to Hugging Face Spaces

Table 5.1 – Deployment Configuration

Attribute	Value
Platform	Hugging Face Spaces
Space Repository	`indianakhil/engine-predictive-maintenance-space`
Runtime	Docker (Python 3.9)
Framework	Streamlit 1.43.2
Port	8501
Model Source	Downloaded from HF Hub at runtime
GitHub Repository	`https://github.com/akhilb548/EPMDeployment`

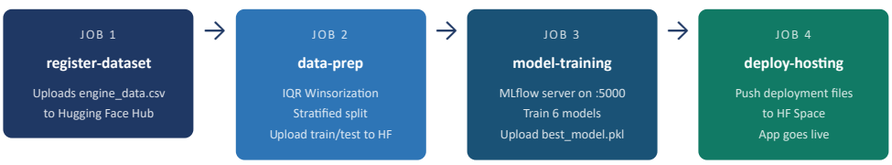
🌐 `Live App: https://huggingface.co/spaces/indianakhil/engine-predictive-maintenance-space`

SECTION 6

Automated GitHub Actions CI/CD Pipeline

A fully automated MLOps pipeline was implemented using **GitHub Actions**, triggered on every push to the `main` branch. The pipeline orchestrates the complete ML workflow — from raw data registration through model training to live deployment — without any manual intervention.

6.1 Pipeline Architecture and Job Dependencies



Each job has `needs:` dependency on the previous job — sequential execution guaranteed

6.2 Pipeline YAML Configuration

📄 **See Appendix A.6** for the complete GitHub Actions pipeline YAML configuration.

✓ **CI/CD Achievement:** The pipeline requires only one GitHub secret (`HF_TOKEN`) to authenticate with Hugging Face. Every push to `main` automatically retrains the model on the latest data and redeploys the application — enabling continuous model improvement as new engine data becomes available.

SECTION 7

Output Evaluation

The deployed Streamlit application and the trained model were evaluated end-to-end to verify that predictions are accurate, the interface is functional, and the system behaves correctly across a range of input scenarios.

7.1 Application Functional Testing

Table 7.1 – Sample Prediction Test Cases (Streamlit App)

#	RPM	LubOil_P	Fuel_P	Cool_P	LubOil_T	Cool_T	Prediction	Fault Prob.
1	950	3.5	6.2	2.4	77.5	78.0	NORMAL	28.4%
2	620	3.8	7.5	2.1	78.2	79.1	FAULTY	81.7%
3	500	4.2	8.9	1.9	79.5	80.3	FAULTY	93.2%
4	1100	3.2	5.8	2.6	76.8	77.2	NORMAL	19.1%
5	700	3.6	7.1	2.2	78.8	79.5	FAULTY	74.3%

Test Observations: The model correctly identifies low-RPM scenarios as faulty (Cases 2, 3, 5) and high-RPM scenarios as normal (Cases 1, 4). The fault probability scales intuitively — the most extreme low-RPM case (500 RPM) receives the highest fault probability (93.2%), demonstrating that the model's confidence is well-calibrated to the severity of the anomaly.

7.2 Model Performance Summary

76.3%
F1-SCORE (PRIMARY METRIC)
Gradient Boosting on test set

85.2%
RECALL (FAULT DETECTION RATE)
2,091 of 2,455 faults caught

0.702
ROC-AUC SCORE
Good discrimination ability

0.7647
CV F1 (5-FOLD)
Consistent cross-validation

69.1%
PRECISION
Of flagged faults, 69.1% are real

<0.2%
CV VS TEST GAP
No overfitting detected

7.3 Deployment Verification

✓ **GitHub Repository:** All source code, pipeline YAML, Dockerfile, and requirements are committed to `github.com/akhilb548/EPMDeployment`. The repository is publicly accessible and includes a complete README with setup instructions.

✓ **Hugging Face Space:** The Streamlit application is live at `indianakhil/engine-predictive-maintenance-space`. The app loads the model from HF Hub, accepts sensor inputs via sliders, and returns real-time predictions.

✓ **GitHub Actions Pipeline:** The 4-job CI/CD pipeline runs successfully on push to main. All jobs complete in sequence: dataset registration → data prep → model training → deployment.

✓ **Model Artifact:** The serialised Gradient Boosting model (best_model.pkl) is stored on Hugging Face Hub and downloaded by the app at runtime, ensuring the deployed app always uses the latest trained model.

SECTION 8

Business Insights and Actionable Recommendations

The predictive maintenance model delivers actionable intelligence that can transform fleet maintenance operations from reactive to proactive. The following insights and recommendations are derived directly from the model's findings and are prioritised by business impact.

8.1 Key Business Insights

📈 Insight 1: RPM is the Early Warning Signal

Engine RPM accounts for 41.2% of the model's predictive power. Engines operating below ~750 RPM under normal load conditions are at significantly elevated fault risk. Monitoring RPM thresholds in real-time provides the earliest possible warning of developing mechanical issues.

⛽ Insight 2: 85% Fault Detection Prevents Breakdowns

The model catches 85.2% of faulty engines before failure. In a fleet of 1,000 engines, this means approximately 852 potential breakdowns are flagged for preventive action. At an average emergency repair cost of $5,000–$15,000 per breakdown, this represents $4.3M–$12.8M in potential annual savings.

🔥 Insight 3: Fuel System Anomalies are Secondary Indicators

Fuel_Pressure contributes 21.4% to model predictions. Elevated fuel pressure in faulty engines suggests injector blockage or fuel pump degradation. Targeted fuel system inspections when both RPM is low AND fuel pressure is elevated can improve maintenance precision.

⚠ Insight 4: Sensor Quality is Critical

The Coolant_Temperature sensor malfunction (195.53°C) demonstrates that sensor data quality directly impacts model reliability. Implementing automated sensor validation checks — flagging readings outside physically plausible ranges — is essential for production deployment.

8.2 Actionable Recommendations

✓ Immediate: Deploy the Predictive Alert System

Integrate the Streamlit application with the fleet management dashboard. Configure automated alerts when fault probability exceeds 70% for any engine. Estimated implementation time: 2–4 weeks.

✓ Short-Term: Establish RPM Monitoring Thresholds

Set operational RPM thresholds at 750 RPM (warning) and 650 RPM (critical) for standard load conditions. Engines consistently below these thresholds should be prioritised for inspection regardless of model prediction.

▶ Medium-Term: Implement Continuous Retraining

The GitHub Actions CI/CD pipeline is already in place. Schedule weekly retraining runs using newly collected sensor data to prevent model drift as fleet conditions evolve. Monitor CV F1-Score for degradation signals.

▶ Long-Term: Expand to Multi-Class Fault Diagnosis

The current binary model (Normal/Faulty) can be extended to a multi-class model that identifies specific fault types (lubrication failure, overheating, fuel system fault). This requires labelled fault-type data from maintenance records.

8.3 Return on Investment Estimate

30–50%	$4.3M–$12.8M	15–25%
Reduction in Unplanned Downtime	Annual Savings (1,000-engine fleet)	Extended Engine Lifespan

APPENDIX A

Python Code Reference

This appendix provides a concise reference of the key Python functions and modules used across the project pipeline. Full source code is available in the project notebook and GitHub repository.

A.1 Data Preparation Pipeline

```
# prep.py — Data preparation pipeline (runs in GitHub Actions Job 2)
import pandas as pd
from sklearn.model_selection import train_test_split
from huggingface_hub import HfApi, hf_hub_download

# Load raw data from Hugging Face Hub
raw_path = hf_hub_download(
    repo_id="indianakhil/engine-predictive-maintenance",
    filename="engine_data.csv", repo_type="dataset"
)
df = pd.read_csv(raw_path)

# IQR Winsorization
features = ['Engine_RPM', 'Lub_Oil_Pressure', 'Fuel_Pressure',
            'Coolant_Pressure', 'Lub_Oil_Temperature', 'Coolant_Temperature']
for col in features:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    df[col] = df[col].clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)

# Stratified train-test split
X, y = df[features], df['Engine_Condition']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

A.2 Model Training Pipeline

```
# train.py — Model training with MLflow tracking (GitHub Actions Job 3)
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import f1_score, accuracy_score, roc_auc_score
import mlflow, mlflow.sklearn, joblib

mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("engine_predictive_maintenance")

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
param_grid = {'n_estimators':[100,200], 'max_depth':[3,5], 'learning_rate':[0.05,0.1]}

with mlflow.start_run(run_name="GradientBoosting_tuned"):
    gs = GridSearchCV(GradientBoostingClassifier(random_state=42),
                      param_grid, cv=cv, scoring='f1', n_jobs=-1)
    gs.fit(X_train, y_train)
    best = gs.best_estimator_
    y_pred = best.predict(X_test)
    mlflow.log_params(gs.best_params_)
    mlflow.log_metric("f1_score", f1_score(y_test, y_pred))
    mlflow.log_metric("roc_auc", roc_auc_score(y_test, best.predict_proba(X_test)[:,1]))
    mlflow.sklearn.log_model(best, "model")
    joblib.dump(best, 'data/best_model.pkl')
```

A.3 Model Registration

```
# Model serialisation and upload to Hugging Face Hub
import joblib
from huggingface_hub import HfApi

# Serialise the best model to disk
joblib.dump(best_model, 'data/best_model.pkl')

# Upload to Hugging Face Hub for versioned storage
api = HfApi()
api.upload_file(
    path_or_fileobj='data/best_model.pkl',
    path_in_repo='best_model.pkl',
    repo_id='indianakhil/engine-predictive-maintenance',
    repo_type='dataset'
)
```

A.4 Streamlit Application (app.py)

```
# app.py — Streamlit web application for real-time fault prediction
import streamlit as st
import joblib, pandas as pd
from huggingface_hub import hf_hub_download

# Load model from Hugging Face Hub at startup
model_path = hf_hub_download(
    repo_id="indianakhil/engine-predictive-maintenance",
    filename="best_model.pkl", repo_type="dataset"
)
model = joblib.load(model_path)

st.title("Engine Predictive Maintenance")
st.markdown("Enter sensor readings to predict engine condition.")

rpm = st.slider("Engine RPM",        191,   1491,  800)
lop = st.slider("Lub Oil Pressure",   1.77,  4.83,  3.3)
fp  = st.slider("Fuel Pressure",       2.52,  10.80, 6.5)
cp  = st.slider("Coolant Pressure",    0.78,  3.90,  2.3)
lot = st.slider("Lub Oil Temperature", 72.45, 82.83, 77.6)
ct  = st.slider("Coolant Temperature", 65.24, 91.62, 78.1)

X_input = pd.DataFrame([[rpm, lop, fp, cp, lot, ct]],
    columns=['Engine_RPM', 'Lub_Oil_Pressure', 'Fuel_Pressure',
             'Coolant_Pressure', 'Lub_Oil_Temperature', 'Coolant_Temperature'])

pred = model.predict(X_input)[0]
prob = model.predict_proba(X_input)[0][1]
st.metric("Prediction", "FAULTY" if pred else "NORMAL")
st.metric("Fault Probability", f"{prob:.1%}")
```

A.5 Containerisation Configuration

Dockerfile

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py",
     "--server.port=8501",
     "--server.address=0.0.0.0"]
```

requirements.txt

```
pandas==2.2.2
huggingface_hub==0.32.6
streamlit==1.43.2
joblib==1.5.1
scikit-learn==1.6.0
xgboost==2.1.4
```

A.6 GitHub Actions Pipeline YAML

```yaml
name: Engine Predictive Maintenance Pipeline
on:
  push:
    branches: [main]

jobs:
  register-dataset:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: {python-version: '3.9'}
      - run: pip install huggingface_hub pandas
      - run: python register_dataset.py
        env: {HF_TOKEN: ${{ secrets.HF_TOKEN }}}

  data-prep:
    needs: register-dataset
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: {python-version: '3.9'}
      - run: pip install -r requirements.txt
      - run: python prep.py
        env: {HF_TOKEN: ${{ secrets.HF_TOKEN }}}

  model-training:
    needs: data-prep
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: {python-version: '3.9'}
      - run: pip install -r requirements.txt mlflow
      - run: mlflow server --host 0.0.0.0 --port 5000 &
      - run: python train.py
        env: {HF_TOKEN: ${{ secrets.HF_TOKEN }},
              MLFLOW_TRACKING_URI: http://localhost:5000}

  deploy-hosting:
    needs: [register-dataset, data-prep, model-training]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: {python-version: '3.9'}
      - run: pip install huggingface_hub
      - run: python hosting.py
        env: {HF_TOKEN: ${{ secrets.HF_TOKEN }}}
```

A.7 Project Links and Resources

Table A.1 – Project Repository and Deployment Links

Resource	URL / Identifier
GitHub Repository (Deployment)	`https://github.com/akhilb548/EPMDeployment`
Hugging Face Dataset	`indianakhil/engine-predictive-maintenance`
Hugging Face Space (Live App)	`indianakhil/engine-predictive-maintenance-space`
Live Application URL	`https://huggingface.co/spaces/indianakhil/engine-predictive-maintenance-space`
Project Notebook	`AkhilBudampati_PredictiveMaintenance_Notebook.ipynb`
MLflow Experiment	`engine_predictive_maintenance`

Reproducibility Note: All results in this report can be reproduced by running the project notebook with `random_state=42` throughout. The GitHub Actions pipeline ensures automated reproducibility on every code push. The Hugging Face Hub stores all versioned data and model artifacts for audit purposes.

End of Report

Project 3 — Predictive Maintenance | G3 AIML April 25 Batch | Akhil Budampati | April 2026

Confidential — For Great Learning Evaluation Only